

Communication entre composants : props, callbacks et lifting state up

Objectifs de cette section

Vous allez apprendre à faire **communiquer plusieurs composants React entre eux**, ce qui est essentiel pour organiser une interface dynamique.
À la fin de cette section, vous serez capable de :

- Passer des données du parent vers l'enfant avec des **props**.
 - Remonter une information de l'enfant vers le parent avec une **fonction callback**.
 - Centraliser un **état partagé** dans un composant parent.
 - Comprendre la notion de **"lifting state up"**.
-

1. Passer des données avec les props

Une **prop** est une information que le composant parent transmet à son enfant.
C'est un **flux descendant** (parent → enfant).

Exemple : afficher une tâche passée en prop

```
function TaskItem({text}) {  
  return <li>{text}</li>;  
}  
  
// Utilisation :  
<TaskItem text="Acheter du pain"/>
```

2. Remonter une action avec une fonction

Pour remonter une interaction (clic, suppression...), on passe une **fonction en prop**.

- L'enfant l'appelle avec les infos nécessaires.
- Le parent la gère comme il veut.

Exemple : un bouton dans l'enfant qui déclenche une fonction du parent

```
function DeleteButton({onDelete}) {  
  return <button onClick={onDelete}>Supprimer</button>;  
}  
  
// Utilisation dans un parent :  
<DeleteButton onDelete={() => alert("Suppression !")}/>
```

3. Partager un state entre plusieurs composants

Quand **deux composants enfants** ont besoin de manipuler ou d'afficher la **même donnée**, on **remonte le state dans le parent**.

C'est ce qu'on appelle **"lifting state up"**.

Exemple : liste de tâches gérée dans le parent, chaque `TaskItem` affiche une tâche

```
import {useState} from "react";

function TaskItem({text, onDelete}) {
  return (
    <li>
      {text}
      <button onClick={onDelete}>Supprimer</button>
    </li>
  );
}

function TaskList() {
  const [tasks, setTasks] = useState(["Faire les courses", "Arroser les plantes"]);

  const handleDelete = (index) => {
    const newTasks = tasks.filter((_, i) => i !== index);
    setTasks(newTasks);
  };

  return (
    <ul>
      {tasks.map((task, index) => (
        <TaskItem key={index} text={task} onDelete={() => handleDelete(index)} />
      ))}
    </ul>
  );
}
```

4. Exercice de mise en pratique

Consignes

1. Créez un composant `TaskItem` qui :
 - Reçoit une prop `text`.
 - Reçoit une prop `onDelete` (callback).
2. Créez un composant `TaskList` qui :
 - Contient un tableau de tâches en state.
 - Affiche une liste de `TaskItem`.
 - Supprime une tâche du tableau lorsqu'un item appelle `onDelete`.
3. Créez un composant `App` qui affiche un `TaskList`.

Organisation attendue

```
/exercices
├─ 13-react-lifting-state-up
│  ├─ App.jsx
│  ├─ TaskList.jsx
│  ├─ TaskItem.jsx
│  └─ main.jsx
```